Actividad en clases: TensorFlow Playground

El propósito de esta actividad es explorar visualmente el funcionamiento de las redes neuronales y comprender cómo su arquitectura, funciones de activación y parámetros de entrenamiento afectan su capacidad para aprender y generalizar.

Utilizaremos **TensorFlow Playground**, una herramienta interactiva que permite construir y entrenar redes neuronales sin necesidad de programarlas.

Instrucciones generales

- Trabaje en TensorFlow Playground, seleccionando el problema de **clasificación (Classification)**.
- Observe tanto las fronteras de decisión como los valores de *training loss* y *test loss*.
- Al comparar configuraciones, procure modificar solamente una característica a la vez y reiniciar el entrenamiento.
- Salvo que se indique lo contrario, utilice la función de activación **tanh**, sin regularización y con un learning rate de 0.03.
- No es necesario alcanzar un valor de pérdida específico. Lo importante es observar el comportamiento de las redes y explicar los resultados.

Recuerde que el entrenamiento depende también de la inicialización aleatoria de los pesos, por lo que una misma arquitectura puede producir resultados diferentes.

Parte A. Problema XOR: ¿por qué necesitamos redes neuronales?

El problema XOR consiste en clasificar puntos de acuerdo con los signos de sus coordenadas:

- Clase $+1$ (azul): x_1 y x_2 tienen el mismo signo.
- Clase -1 (naranjo): x_1 y x_2 tienen signos opuestos.

Experimento 1: Modelo lineal

Seleccione el dataset XOR y utilice solamente las features x_1 y x_2.

1. Construya un modelo sin capas ocultas y entrénelo. ¿Consigue separar correctamente las dos clases? Explique por qué.

   No logra separar las clases, porque sin capas ocultas el modelo es lineal y su frontera de decisión es una recta, mientras que en XOR las clases ocupan cuadrantes opuestos y ninguna recta puede dejar todos los puntos azules de un lado y los naranjos del otro. Por eso el loss se estanca cerca de 0.5.

2. Agregue la feature $x_1 x_2$, manteniendo el modelo sin capas ocultas. ¿Qué ocurre ahora? ¿Por qué esta nueva característica permite mejorar la clasificación?

   Al agregar x1x2 el modelo clasifica casi perfectamente, con el loss cerca de 0 y una frontera que separa los cuadrantes correctamente. Esto ocurre porque la clase depende del signo del producto (positivo si x_1 y x_2 tienen el mismo signo, negativo si son opuestos), de modo que en ese nuevo espacio de features basta una frontera lineal en $x_1 x_2 = 0$.

Experimento 2: Capas ocultas y funciones de activación

Vuelva a seleccionar solamente x_1 y x_2.

3. Agregue una capa oculta con función de activación $tanh$. Experimente con distintas cantidades de neuronas hasta encontrar una arquitectura sencilla que consiga separar correctamente las clases. ¿Cuántas neuronas necesitó? Registre los valores de *training loss* y *test loss*.

   Se necesitaron 3 neuronas en una sola capa oculta para separar correctamente las clases de forma consistente y simple. Se registró un training loss de 0.08 y un test loss de 0.03, ambos cercanos a 0.

4. Manteniendo esa arquitectura, cambie la función de activación a **Linear** y vuelva a entrenar. ¿Qué ocurre con la frontera de decisión? ¿Por qué?

   Con activación de Linear la frontera de decisión se vuelve un degradado casi plano y el modelo no separa las clases, con el loss estancado en 0.5 aprox. Esto ocurre porque una composición de capas lineales equivale matemáticamente a una sola transformación lineal, por lo que las capas ocultas no aportan poder expresivo.

5. Compare los resultados de agregar la feature $x_1 x_2$ con los de utilizar una capa oculta. ¿Qué diferencia existe entre ambas estrategias?

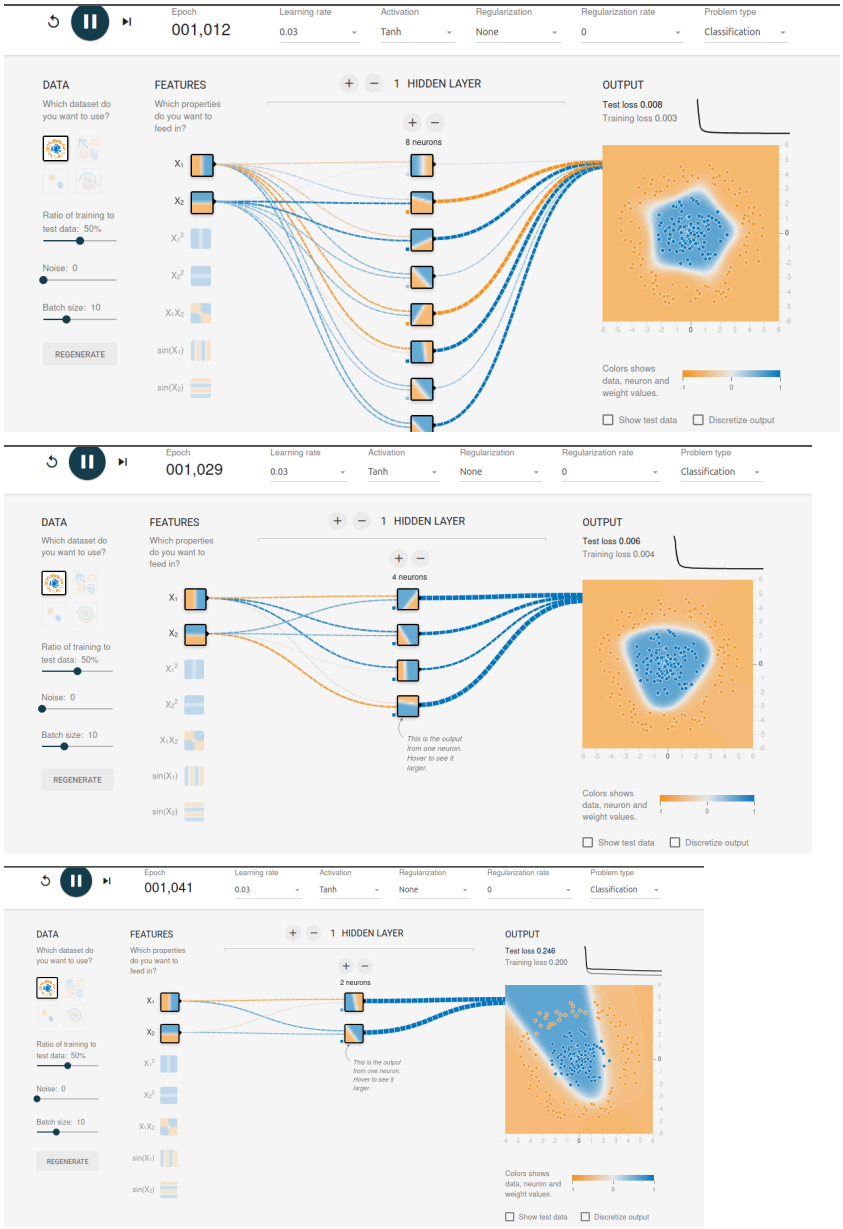
Agregar la feature x1x2 resuelve el problema con un modelo lineal simple, porque se le está entregando a la red neuronal la transformación correcta, lo que requiere conocer de antemano la estructura del problema. En cambio, la capa oculta aprende por sí misma esa representación a partir de los datos, lo que es más general y no exige conocimiento previo.

Parte B. Complejidad de la red: círculos concéntricos

Seleccione el dataset de círculos concéntricos (**Circle**) y utilice solamente las features x_1 y x_2.

Experimento 3: Número de neuronas

1. Construya una red con una sola capa oculta y 2 neuronas. Entrénela y observe la frontera de decisión.



2. Repita el experimento utilizando 4 y luego 8 neuronas, manteniendo los demás parámetros fijos. Compare los resultados después de un número similar de épocas de entrenamiento.

   Con 2 neuronas la red no logra encerrar el círculo y queda con un loss alto (0.2 aprox.), mientras que con 4 y 8 neuronas la frontera rodea correctamente a los puntos azules y el loss baja considerablemente a 0.005 aprox en ambos casos.

3. ¿Cómo cambia la frontera de decisión al aumentar la cantidad de neuronas?

Al aumentar las neuronas, la frontera pasa de una forma alargada y poco ajustada (2 neuronas) a una región cerrada que rodea el círculo (4 y 8), y con 8 se vuelve algo más suave y con más lados.

4. ¿Agregar más neuronas siempre produce una mejor clasificación? Compare los valores de *training loss* y *test loss*.

   No siempre: pasar de 2 a 4 neuronas mejoró mucho el training loss (0.200 → 0.004) y el test loss (0.246 → 0.006), pero pasar de 4 a 8 no mejoró de forma apreciable (training 0.003 y test 0.008, incluso algo peor en test). Una vez que la red tiene capacidad suficiente para el problema, agregar más neuronas solo añade parámetros sin beneficio claro y aumenta el riesgo de sobreajuste.

5. ¿Qué relación existe entre la cantidad de neuronas y el poder expresivo (*expressive power*) de una red neuronal?

   Más neuronas aumentan el poder expresivo, porque la red puede combinar más funciones simples y representar fronteras más complejas. Sin embargo, ese poder expresivo tiene un límite útil: más allá de lo que el problema requiere no se traduce en mejor generalización.

Parte C. Un problema más complejo: espiral

Seleccione el dataset **Spiral**, que presenta una frontera de clasificación considerablemente más compleja.

Experimento 4: Arquitectura y selección de features

1. Utilizando solamente las features x_1 y x_2, experimente con el número de capas ocultas y neuronas para obtener una clasificación razonable. ¿Qué arquitectura utilizó? Registre el *training loss* y el *test loss*.

   Se utilizaron 3 capas ocultas con 8, 6 y 4 neuronas (tanh) con solo x1 y x2, alcanzando un training loss de 0.004 y un test loss de 0.007 tras 1500 épocas aprox.

2. Ahora active las features adicionales que ofrece Playground y vuelva a entrenar. ¿Puede conseguir resultados similares utilizando una arquitectura más sencilla?

   Sí, con las features sin(x1) y sin(x2) bastó una arquitectura más simple, de 2 capas ocultas con 4 neuronas cada una (18 neuronas en total en el punto 1

contra 8 en el punto 2). Los losses fueron algo mayores (training 0.012, test 0.016), pero la clasificación sigue siendo razonable.

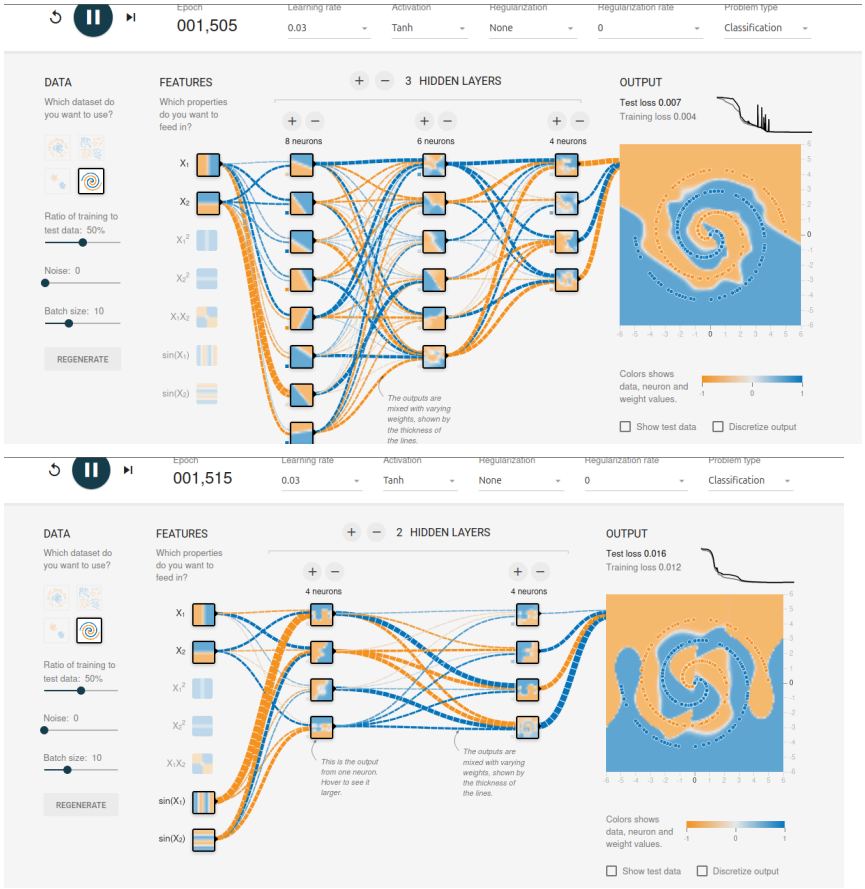
3. Compare ambas soluciones. ¿Qué ventajas y desventajas observa al proporcionar características transformadas en lugar de utilizar solamente las variables originales?

   La ventaja de entregar features transformadas es que la red necesita menos capas y neuronas, porque parte de la complejidad ya viene resuelta en la entrada, lo que reduce el costo de entrenamiento. La desventaja es que depende del conocimiento del problema y de elegir bien las transformaciones: aquí produjo una frontera menos ajustada (con “islas” y regiones irregulares en los bordes) y losses algo mayores, mientras que la red más profunda con solo x1 y x2 aprendió la representación por sí misma y ajustó mejor.

4. Examine las fronteras de decisión y los valores de pérdida en entrenamiento y test. ¿Observa señales de *underfitting* u *overfitting*? Justifique su respuesta.

   No se observa underfitting porque ambos training losses son bajos (0.004 y 0.012), ni overfitting claro porque el test loss es muy cercano al training loss en ambos casos (0.007 y 0.016). Sin embargo, la frontera del punto 2 muestra regiones azules y naranjas en zonas sin datos, lo que indica que la red extrapola de forma arbitraria fuera de la espiral, algo que conviene mencionar como una posible señal leve de sobreajuste en la forma de la frontera.

Nota: No es necesario conseguir una clasificación perfecta. El objetivo es investigar cómo las características de entrada y la arquitectura afectan el aprendizaje.



Parte D. Entrenamiento: efecto del learning rate

Experimento 5: ¿Un learning rate mayor significa aprender más rápido?

Seleccione nuevamente el dataset de círculos y utilice una arquitectura que haya funcionado razonablemente bien en la Parte B.

Mantenga fija la arquitectura, las features y la función de activación.

Se usó el dataset Circle, con solo x1 y x1, 1 capa oculta con 4 neuronas, activación tanh y sin regularización.

1. Entrene la red utilizando tres valores de learning rate: 0.003, 0.03 y 0.3.
2. Reinicie el entrenamiento en cada caso y observe la evolución de la función de pérdida durante un número similar de épocas.

Learning rate	Épocas	Training loss	Test loss	Observación de la curva

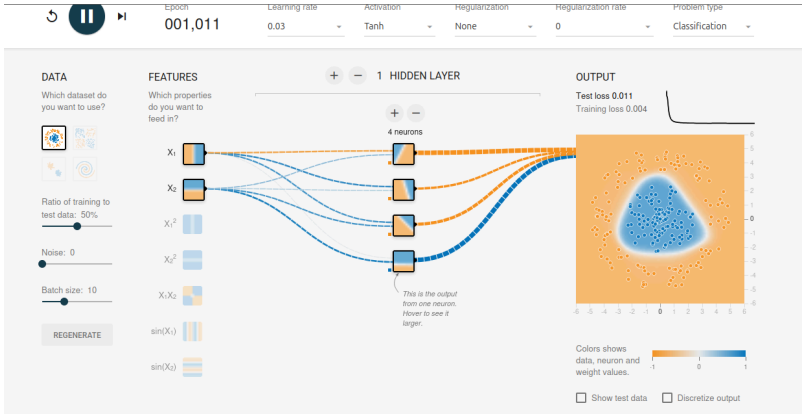
0.003	1009	0.033	0.030	Descenso lento y suave; aún no converge del todo
0.03	1011	0.004	0.011	Descenso rápido y estable
0.3	1012	0.000	0.001	Descenso muy rápido, con pequeñas oscilaciones al inicio

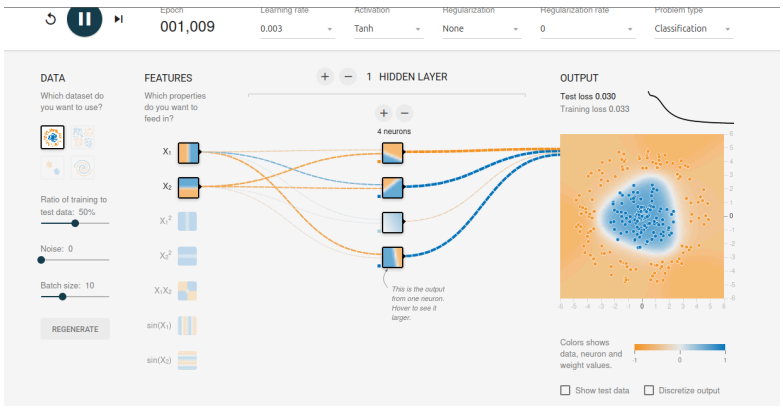
3. ¿Qué diferencias observa en la rapidez con que disminuye la pérdida?

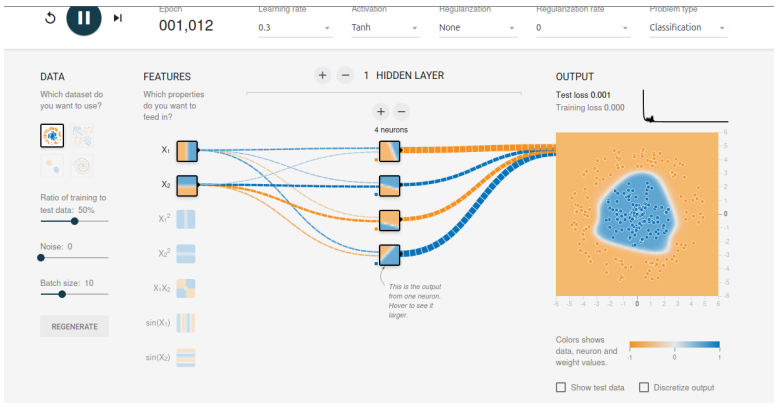
   Con Learning rate de 0.003 la pérdida baja lentamente y tras 1000 épocas aprox. sigue relativamente alta (0.033), con 0.03 baja rápido y se estabiliza (0.004), y con 0.3 cae casi de inmediato a 0.0. Un Learning rate mayor da pasos más grandes en cada actualización, por lo que se acerca antes al mínimo.

4. ¿Un learning rate mayor produce siempre mejores resultados? ¿Qué problemas pueden aparecer cuando es demasiado grande?

   No siempre: en este experimento cuando su valor era de 0.3 dio el mejor resultado, pero eso depende del problema, y un Learning rate demasiado grande puede hacer que los pasos “salten” sobre el mínimo, provocando oscilaciones, pérdida inestable o incluso divergencia. En la curva de 0.3 ya se ven pequeñas oscilaciones al inicio, una señal temprana de este riesgo que en esta red sencilla no llegó a afectar el resultado.







Desafío opcional

Explore si es posible mejorar significativamente la clasificación con un modelo sin capas ocultas utilizando solamente dos de las features transformadas disponibles x_1^2, x_2^2, $x_1 x_2$, $sin(x_1)$, $sin(x_2)$.

¿Qué combinaciones probó? ¿Qué aprendió sobre la importancia del *feature engineering*?

Conclusiones

A partir de los experimentos realizados, discuta brevemente:

1. ¿Por qué son importantes las funciones de activación no lineales en las redes neuronales?

   Las funciones de activación no lineales permiten que la red aprenda fronteras de decisión curvas y complejas. Sin ellas, como vimos en el experimento con activación Linear, varias capas equivalen a un único modelo lineal que no puede resolver problemas como XOR.

2. ¿Una red con más capas o neuronas necesariamente aprende mejor?

   No necesariamente: en los círculos, pasar de 4 a 8 neuronas no mejoró los resultados (training 0.004 → 0.003, test 0.006 → 0.008), porque con 4 ya había capacidad suficiente. Más capas o neuronas aumentan el poder expresivo, pero lo importante es que la arquitectura sea adecuada para el problema, ya que en la espiral la solución con features transformadas y menos neuronas también funcionó razonablemente bien.

3. El Teorema del Aproximador Universal establece que una red neuronal suficientemente grande puede aproximar funciones continuas con una precisión arbitraria. ¿Significa esto que siempre podremos entrenar exitosamente esa red? Explique.

   No, porque que una red pueda representar una función no garantiza que el entrenamiento logre encontrarla. En nuestros experimentos el resultado dependió de la inicialización aleatoria, del Learning rate y de la arquitectura, por lo que una red lo bastante grande puede no entrenarse bien si estos factores no son adecuados.

Registro de resultados

Para cada parte, registre las configuraciones utilizadas, los resultados más relevantes y sus conclusiones. Incluya capturas de pantalla de las soluciones que considere más interesantes y el enlace de Playground correspondiente, que permite compartir la configuración de la red.

No es necesario incluir todas las pruebas realizadas, solamente las necesarias para justificar sus respuestas.